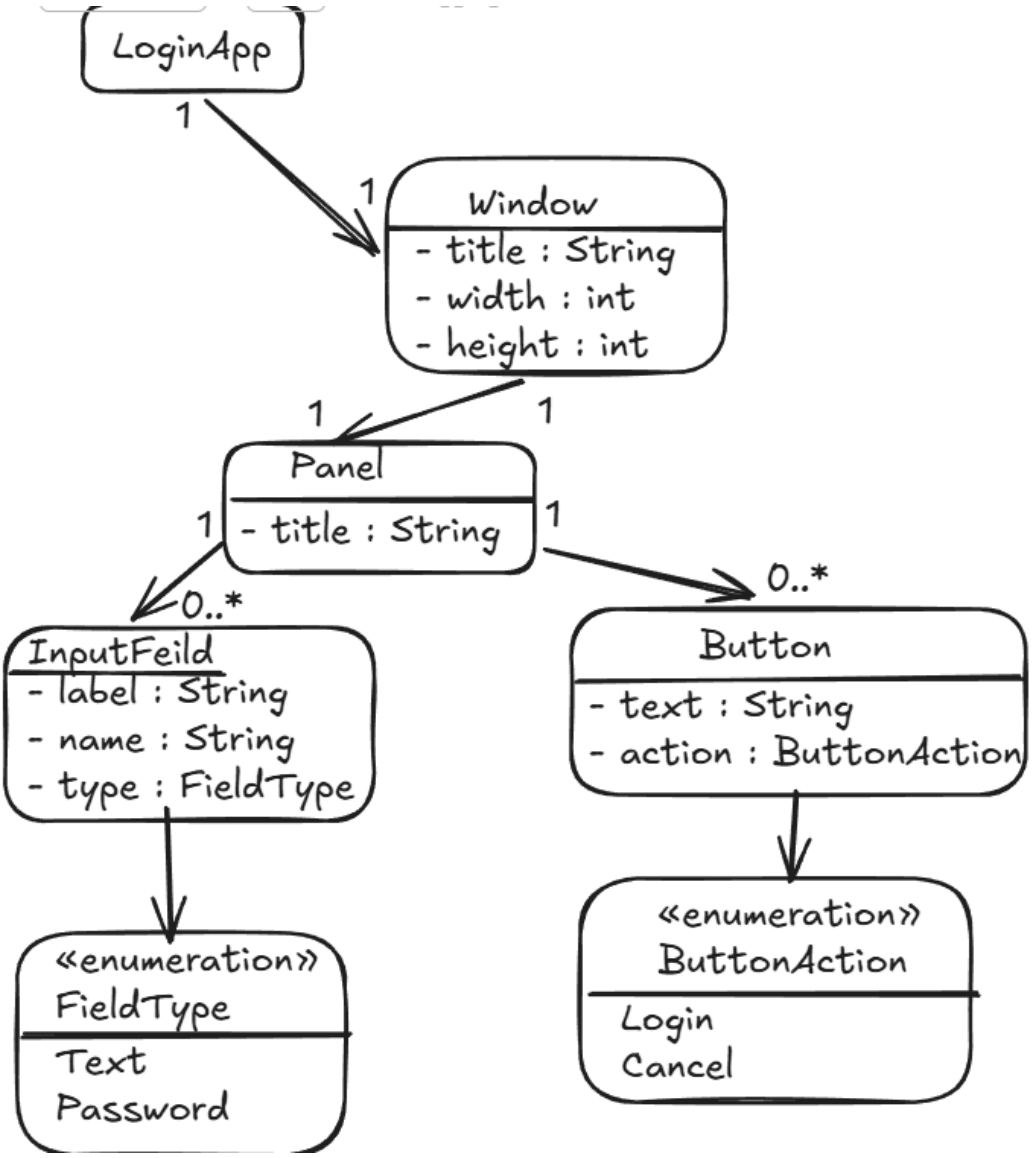
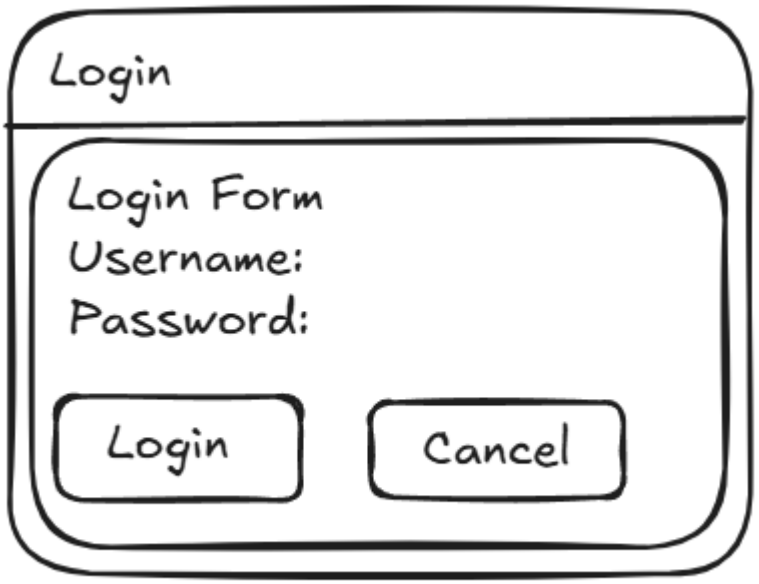
MBSD_Project

https://github.com/rahimamunni97/MBSD-LoginGUI-DSL.git

Metamodel


LoginApp
1
1
Window
- title : String
- width : int
- height : int
1
1
Panel
- title : String
1
1
0..*
0..*
InputFeild
- label : String
- name : String
- type : FieldType
Button
- text : String
- action : ButtonAction
«enumeration»
FieldType
Text
Password
«enumeration»
ButtonAction
Login
Cancel


Login
Login Form
Username:
Password:
Login
Cancel

DSL Syntax

LoginApp
Window title = "Login"
Window width = 350
Window height = 220

Panel title = "Login Form"

InputField
label = "Username:"
name = "username"
type = TEXT

InputField
label = "Password:"
name = "password"
type = PASSWORD

Button
text = "Login"
action = LOGIN

Button
text = "Cancel"
action = CANCEL

Internal DSL Syntax

Internal DSL Syntax:

```
LoginApp.window("Login", 350, 220)
        .panel("Login Form")
        .field("Username:", "username", "text")
        .field("Password:", "password", "password")
        .button("Login", "login")
        .button("Cancel", "cancel")
        .show();
```

The internal DSL is implemented in Java using a fluent API.
The technique used is method chaining.
Each method call adds one part of the GUI model: window, panel, input fields, and buttons.
The final show() method executes the model and displays the GUI using Java Swing.

Internal DSL Implementation

Example Internal DSL

```
public class Main {
    public static void main(String[] args) {
        LoginApp.window("Login")
                .panel("Login Form")
                .field("Username:", "username", "text")
                .field("Password:", "password", "password")
                .button("Login", "login")
                .button("Cancel", "cancel")
                .show();
    }
}
```

Main.java:

```
package app;

public class Main {

    public static void main(String[] args) {

        LoginApp.window("Login", 350, 220)
                .panel("Login Form")
                .field("Username:", "username", "text")
                .field("Password:", "password", "password")
                .button("Login", "login")
                .button("Cancel", "cancel")
                .show();
    }
}
```

LoginApp.java:

```
package app;

import javax.swing.*;
import java.awt.*;
import java.util.ArrayList;
import java.util.List;

public class LoginApp {

    public static WindowBuilder window(String title, int width, int height) {
        return new WindowBuilder(title, width, height);
    }
}

class WindowBuilder {
```

```
    private final String title;
    private final int width;
    private final int height;
    private PanelBuilder panel;

    public WindowBuilder(String title, int width, int height) {
        this.title = title;
        this.width = width;
        this.height = height;
    }

    public PanelBuilder panel(String title) {
        this.panel = new PanelBuilder(this, title);
        return panel;
    }

    public void show() {
        JFrame frame = new JFrame(title);
        frame.setSize(width, height);
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        frame.setLocationRelativeTo(null);

        if (panel != null) {
            frame.add(panel.createPanel());
        }

        frame.setVisible(true);
    }
}

class PanelBuilder {

    private final WindowBuilder window;
    private final String title;

    private final List<String[]> fields = new ArrayList<>();
    private final List<String[]> buttons = new ArrayList<>();

    public PanelBuilder(WindowBuilder window, String title) {
        this.window = window;
        this.title = title;
    }

    public PanelBuilder field(String label, String name, String type) {
        fields.add(new String[] { label, name, type });
        return this;
    }
```

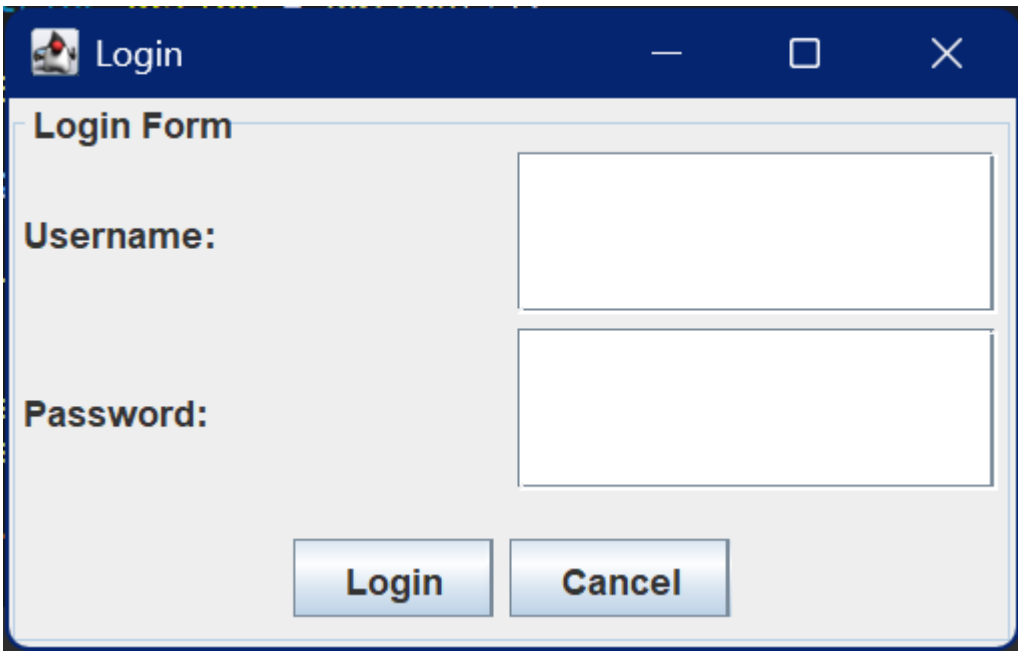
```
public PanelBuilder button(String text, String action) {
    buttons.add(new String[] { text, action });
    return this;
}

public void show() {
    window.show();
}

public JPanel createPanel() {

    JPanel mainPanel = new JPanel(new BorderLayout(10, 10));
    mainPanel.setBorder(BorderFactory.createTitledBorder(title));

    JPanel fieldPanel = new JPanel(
            new GridLayout(fields.size(), 2, 5, 5));

    for (String[] field : fields) {

        fieldPanel.add(new JLabel(field[0]));

        if (field[2].equalsIgnoreCase("password")) {
            fieldPanel.add(new JPasswordField(15));
        } else {
            fieldPanel.add(new JTextField(15));
        }
    }

    JPanel buttonPanel = new JPanel(new FlowLayout());

    for (String[] button : buttons) {

        JButton jButton = new JButton(button[0]);

        String action = button[1];

        jButton.addActionListener(e ->
                System.out.println(action + " button clicked"));

        buttonPanel.add(jButton);
    }

    mainPanel.add(fieldPanel, BorderLayout.CENTER);
    mainPanel.add(buttonPanel, BorderLayout.SOUTH);

    return mainPanel;
}
```

}

Implementation technique: Builder pattern with method chaining.

Why: It makes the internal DSL readable and close to natural language. Each method creates or adds part of the GUI model.

Execution: The show() method executes the metamodel instance by creating a Java Swing JFrame, JPanel, labels, text fields, password fields, and buttons.



External DSL example

External DSL program example:

```
loginApp {
   window "Login" size 350 220 {
      panel "Login Form" {
         field "Username:" name username type text
         field "Password:" name password type password
         button "Login" action login
         button "Cancel" action cancel
      }
   }
}
```

Explanation:

loginApp → Application definition
window → Creates a window
"Login" → Window title
size 350 220 → Window width and height
panel → Creates a panel inside the window
field → Creates an input field
name → Internal field identifier
type → Field type (text or password)
button → Creates a button
action → Action associated with the button

EBNF Specification

```
Program = "loginApp", "{", Window, "}";

Window = "window", String, "size", Number, Number, "{", Panel, "}";

Panel = "panel", String, "{", { Component }, "}";

Component = Field | Button;

Field = "field", String, "name", Identifier, "type", FieldType;

Button = "button", String, "action", ButtonAction;

FieldType = "text" | "password";

ButtonAction = "login" | "cancel";

Identifier = Letter, { Letter | Digit | "_" };

Number = Digit, { Digit };

String = "", { Character }, "";

Letter = "A" | "B" | "C" | "D" | "E" | "F" | "G"
       | "H" | "I" | "J" | "K" | "L" | "M" | "N"
       | "O" | "P" | "Q" | "R" | "S" | "T" | "U"
       | "V" | "W" | "X" | "Y" | "Z"
       | "a" | "b" | "c" | "d" | "e" | "f" | "g"
       | "h" | "i" | "j" | "k" | "l" | "m" | "n"
       | "o" | "p" | "q" | "r" | "s" | "t" | "u"
       | "v" | "w" | "x" | "y" | "z";

Digit = "0" | "1" | "2" | "3" | "4" | "5" | "6" | "7" | "8" | "9";

Character = ? any character except " ?;
```

Xtext grammar

```
grammar org.example.login.LoginDsl with org.eclipse.xtext.common.Terminals

generate loginDsl "http://www.example.org/loginDsl"

Program:
    'loginApp' '{'
        window=Window
    '}'
;

Window:
    'window' title=STRING 'size' width=INT height=INT '{'
        panel=Panel
    '}'
;

Panel:
    'panel' title=STRING '{'
        components+=Component*
    '}'
;

Component:
    Field | Button
;

Field:
    'field' label=STRING
    'name' name=ID
    'type' type=FieldType
;

Button:
    'button' text=STRING
    'action' action=ButtonAction
;

enum FieldType:
    text='text' |
    password='password'
;

enum ButtonAction:
    login='login' |
    cancel='cancel'
;
```

The grammar follows the metamodel directly. A program contains one window. The window contains one panel. The panel contains fields and buttons. Fields have a name and type, while buttons have an action. The grammar is intentionally simple because the DSL is designed for login form prototyping.

Example valid DSL:

```
loginApp {
   window "Login" size 350 220 {

      panel "Login Form" {
         field "Username:" name username type text
         field "Password:" name password type password

         button "Login" action login
         button "Cancel" action cancel
      }

      panel "Actions" {
         button "Login" action login
      }
   }
}
```

External DSL Code generation

Example Program 1: Student Login Form

```
package app;

public class Main {

    public static void main(String[] args) {

        LoginApp.window("Student Login", 400, 250)
                .panel("Student Portal")
                .field("Student ID:", "studentId", "text")
                .field("Password:", "password", "password")
                .button("Login", "login")
                .button("Cancel", "cancel")
                .show();
    }
}
```

Explanation:

This DSL program creates a login window.

The window title is "Login" and its size is 350 × 220.
The window contains one panel called "Login Form".

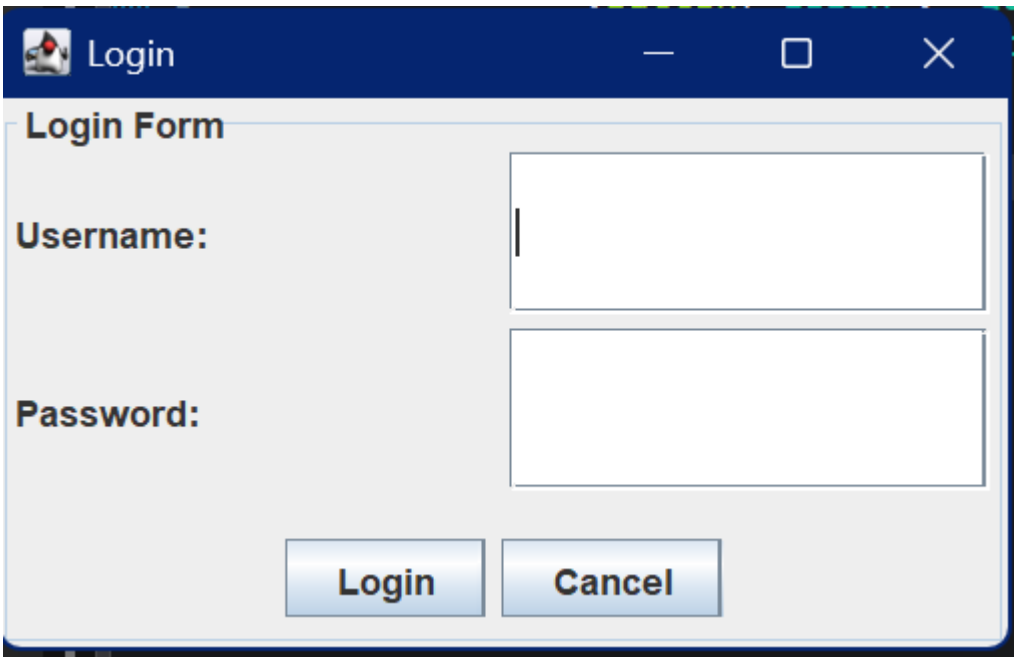
Inside the panel:
- A text field is created for Username.
- A password field is created for Password.
- A Login button is created.
- A Cancel button is created.

The code generator translates this DSL program into Java Swing code and displays a graphical login form.

Generated Output:

Window Title: Login
Window Size: 350 × 220

Components:
- Username Label
- Username Text Field
- Password Label
- Password Field
- Login Button
- Cancel Button



Example 2: Student Login

```
loginApp {
    window "Student Login" size 400 250 {
        panel "Student Portal" {
            field "Student ID:" name studentId type text
            field "Password:" name password type password
            button "Login" action login
            button "Cancel" action cancel
        }
    }
}
```

Explanation:

This DSL program creates a student login window.

The window title is "Student Login" and its size is 400 × 250.
The panel is named "Student Portal".

Inside the panel:
- A text field is created for Student ID.
- A password field is created for Password.
- A Login button is created.
- A Cancel button is created.

The generated Java Swing application displays a student login form.

Generated Output:

Window Title: Student Login
Window Size: 400 × 250

Components:
- Student ID Label
- Student ID Text Field
- Password Label
- Password Field
- Login Button
- Cancel Button

Code Generator

Code Generator

The code generator takes a program written in the Login GUI DSL and produces Java Swing code.

The generator reads:
- window title
- window width and height
- panel title
- input fields
- buttons

Then it creates Java Swing components:
- JFrame for window
- JPanel for panel
- JLabel for field labels
- JTextField for text fields
- JPasswordField for password fields
- JButton for buttons

Finally, the generated Java program can be executed to display the login GUI.

QuickGUIGenerator.xtend

```
package org.example.quickgui2.generator
import org.eclipse.emf.ecore.resource.Resource
import org.eclipse.xtext.generator.AbstractGenerator
import org.eclipse.xtext.generator.IFileSystemAccess2
import org.eclipse.xtext.generator.IGeneratorContext
import org.example.quickgui2.quickGUI.Program
import org.example.quickgui2.quickGUI.Field
import org.example.quickgui2.quickGUI.Button
import org.example.quickgui2.quickGUI.FieldType
import org.example.quickgui2.quickGUI.ButtonAction
class QuickGUIGenerator extends AbstractGenerator {
override void doGenerate(Resource resource, IFileSystemAccess2 fsa, IGeneratorContext context) {
  val program = resource.allContents.filter(Program).head
  if (program !== null) {
     fsa.generateFile("GeneratedLogin.java", program.compile)
  }
}
def compile(Program program) '''
  import javax.swing.*;
  import java.awt.*;
  public class GeneratedLogin {
    public static void main(String[] args) {
      JFrame frame = new JFrame("«program.window.title»");
      frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
      frame.setSize(350, 220);
      frame.setLocationRelativeTo(null);
      JPanel mainPanel = new JPanel();
```

```
    mainPanel.setBorder(
        BorderFactory.createTitledBorder(
            "«program.window.panel.title»"
        )
    );
    mainPanel.setLayout(
        new GridLayout(0, 2, 10, 10)
    );
    «FOR c : program.window.panel.components»
      «IF c instanceof Field»
        «val f = c as Field»
        JLabel «f.name»Label =
          new JLabel("«f.label»");
        «IF f.type == FieldType.PASSWORD»
          JPasswordField «f.name»Field =
            new JPasswordField(15);
        «ELSE»
          JTextField «f.name»Field =
            new JTextField(15);
        «ENDIF»
        mainPanel.add(«f.name»Label);
        mainPanel.add(«f.name»Field);
      «ENDIF»
    «ENDFOR»
    «FOR c : program.window.panel.components»
      «IF c instanceof Button»
        «val b = c as Button»
        JButton «b.action.toString»Button =
          new JButton("«b.text»");
        «IF b.action == ButtonAction.LOGIN»
          «b.action.toString»Button.addActionListener(e ->
            JOptionPane.showMessageDialog(
                frame,
                "Login button clicked"
            )
          );
        «ELSE»
          «b.action.toString»Button.addActionListener(e ->
            frame.dispose()
          );
        «ENDIF»
        mainPanel.add(
          «b.action.toString»Button
        );
      «ENDIF»
    «ENDFOR»
    frame.add(mainPanel);
    frame.setVisible(true);
  }
}
'''
}
```

The current generator uses a fixed default window size of 350 × 220, although the metamodel and grammar include width and height. In a later version, the generator can use the width and height values directly from the DSL.

Assignment

Code Generator for Login GUI DSL

Purpose

This code generator is the first version of the generator for my external Login GUI DSL. The goal of the generator is to translate a DSL program into Java Swing code.

The DSL describes a simple login GUI using domain concepts such as:

- window
- panel
- field
- button

The generated Java code creates an executable Swing GUI.

Example DSL Program

```
loginApp {
   window "Login" {
      panel "Login Form" {
         field "Username:" name username type text
         field "Password:" name password type password
         button "Login" action login
         button "Cancel" action cancel
      }
   }
}
```

Expected Generated Output

The code generator should create a Java file named:

GeneratedLogin.java

The generated Java program should display:

- A JFrame with title `Login`
- A JPanel with title `Login Form`
- A username label and text field
- A password label and password field
- A Login button

- A Cancel button

The main mapping is:

Window → JFrame
Panel → JPanel
Field text → JTextField
Field password → JPasswordField
Button → JButton

Code Generator File: QuickGUIGenerator.xtend

```
package org.example.quickgui2.generator

import org.eclipse.emf.ecore.resource.Resource
import org.eclipse.xtext.generator.AbstractGenerator
import org.eclipse.xtext.generator.IFileSystemAccess2
import org.eclipse.xtext.generator.IGeneratorContext
import org.example.quickgui2.quickGUI.Program
import org.example.quickgui2.quickGUI.Field
import org.example.quickgui2.quickGUI.Button
import org.example.quickgui2.quickGUI.FieldType
import org.example.quickgui2.quickGUI.ButtonAction

class QuickGUIGenerator extends AbstractGenerator {

   override void doGenerate(Resource resource, IFileSystemAccess2 fsa,
IGeneratorContext context) {
      val program = resource.allContents.filter(Program).head

      if (program !== null) {
         fsa.generateFile("GeneratedLogin.java", program.compile)
      }
   }

   def compile(Program program) '''
      import javax.swing.*;
      import java.awt.*;

      public class GeneratedLogin {

         public static void main(String[] args) {

            JFrame frame = new JFrame("«program.window.title»");
            frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
            frame.setSize(350, 220);
            frame.setLocationRelativeTo(null);
```

```
JPanel mainPanel = new JPanel();
mainPanel.setBorder(
   BorderFactory.createTitledBorder(
      "«program.window.panel.title»"
   )
);

mainPanel.setLayout(
   new GridLayout(0, 2, 10, 10)
);

«FOR c : program.window.panel.components»
   «IF c instanceof Field»
      «val f = c as Field»

      JLabel «f.name»Label =
         new JLabel("«f.label»");

      «IF f.type == FieldType.PASSWORD»
         JPasswordField «f.name»Field =
            new JPasswordField(15);
      «ELSE»
         JTextField «f.name»Field =
            new JTextField(15);
      «ENDIF»

      mainPanel.add(«f.name»Label);
      mainPanel.add(«f.name»Field);

   «ENDIF»
«ENDFOR»

«FOR c : program.window.panel.components»
   «IF c instanceof Button»
      «val b = c as Button»

      JButton «b.action.toString»Button =
         new JButton("«b.text»");

      «IF b.action == ButtonAction.LOGIN»
         «b.action.toString»Button.addActionListener(e ->
            JOptionPane.showMessageDialog(
               frame,
               "Login button clicked"
            )
         );
      «ELSE»
```

```
                «b.action.toString»Button.addActionListener(e ->
                    frame.dispose()
                );
            «ENDIF»

            mainPanel.add(
                «b.action.toString»Button
            );

        «ENDIF»
    «ENDFOR»

    frame.add(mainPanel);
    frame.setVisible(true);
  }
 }
 '''
}
```

Explanation of the Generator

The generator starts from the root model element `Program`.

val program = resource.allContents.filter(Program).head

Then it generates one Java file:

fsa.generateFile("GeneratedLogin.java", program.compile)

The `compile` method contains a Java Swing template.
Values from the DSL are inserted into the generated Java code.

For example:

«program.window.title»

is used as the title of the generated JFrame.

The generator loops through all components inside the panel:

FOR c : program.window.panel.components

If the component is a field, the generator creates either:

JTextField

or:

JPasswordField

If the component is a button, the generator creates a:

JButton

The Login button displays a message dialog, while the Cancel button closes the window.

Current Status

This is the first version of the code generator.
It is simple, but it shows the intended transformation from the external DSL to Java Swing code.

The generator demonstrates how model elements from the DSL can be mapped to executable Java GUI components.

Individual Extension

Individual Extension: Validation

For my individual extension, I chose validation.

The purpose of the validation is to prevent simple mistakes in DSL programs before code generation.

The validation rules are:

1. Window title cannot be empty.
2. Field name cannot be empty.
3. Button text cannot be empty.

These rules improve the quality of DSL programs and help avoid invalid GUI definitions.

Individual Extension Implementation: Validation

Individual Extension Implementation: Validation

For my individual extension, I implemented validation for the external Login GUI DSL.

The purpose of validation is to check the DSL program before code generation. This helps prevent invalid GUI definitions and avoids problems in the generated Java Swing code.

I implemented the validation in the file:

QuickGUIValidator.java

Validation Rules

The validator checks three simple rules:

1. The window title cannot be empty.
2. The field name cannot be empty.
3. The button text cannot be empty.

Validator Code

```
package org.example.quickgui2.validation;

import org.eclipse.xtext.validation.Check;
import org.example.quickgui2.quickGUI.Window;
import org.example.quickgui2.quickGUI.Field;
import org.example.quickgui2.quickGUI.Button;
import org.example.quickgui2.quickGUI.QuickGUIPackage;

public class QuickGUIValidator extends AbstractQuickGUIValidator {

    @Check
    public void checkWindowTitle(Window window) {
        if (window.getTitle().trim().isEmpty()) {
            error(
                "Window title cannot be empty",
                QuickGUIPackage.Literals.WINDOW__TITLE
            );
        }
    }

    @Check
    public void checkFieldName(Field field) {
        if (field.getName().trim().isEmpty()) {
            error(
```

```
            "Field name cannot be empty",
            QuickGUIPackage.Literals.FIELD__NAME
        );
      }
   }

   @Check
   public void checkButtonText(Button button) {
      if (button.getText().trim().isEmpty()) {
         error(
            "Button text cannot be empty",
            QuickGUIPackage.Literals.BUTTON__TEXT
         );
      }
   }
}
```

Invalid DSL Example

```
loginApp {
   window "" {
      panel "Login Form" {
         field "Username:" name username type text
         button "" action login
      }
   }
}
```

Expected Validation Result

The validator reports errors because:

window ""

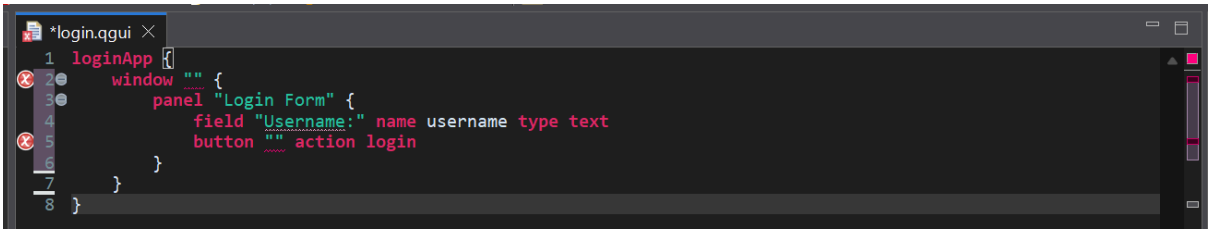
has an empty window title, and:

button ""

has an empty button text.

Expected error messages:

Window title cannot be empty
Button text cannot be empty



Explanation

This extension improves the DSL because invalid GUI programs are detected before code generation.

Instead of generating Java Swing code from an incomplete GUI definition, the validation step gives feedback to the DSL user.

This follows the idea from the course that validation reduces invalid metamodel instances without making the grammar more complicated.